Module 3: Pretraining Data Pipeline

From Raw Documents to Verified Token Batches

BRAIN team

IMT Atlantique · January 2027

IMT Atlantique
Bretagne-Pays de la Loire
École Mines-Télécom

BRAIN

Lecture map

1. From documents to tokens
2. Packing documents into examples
3. Storage and correctness

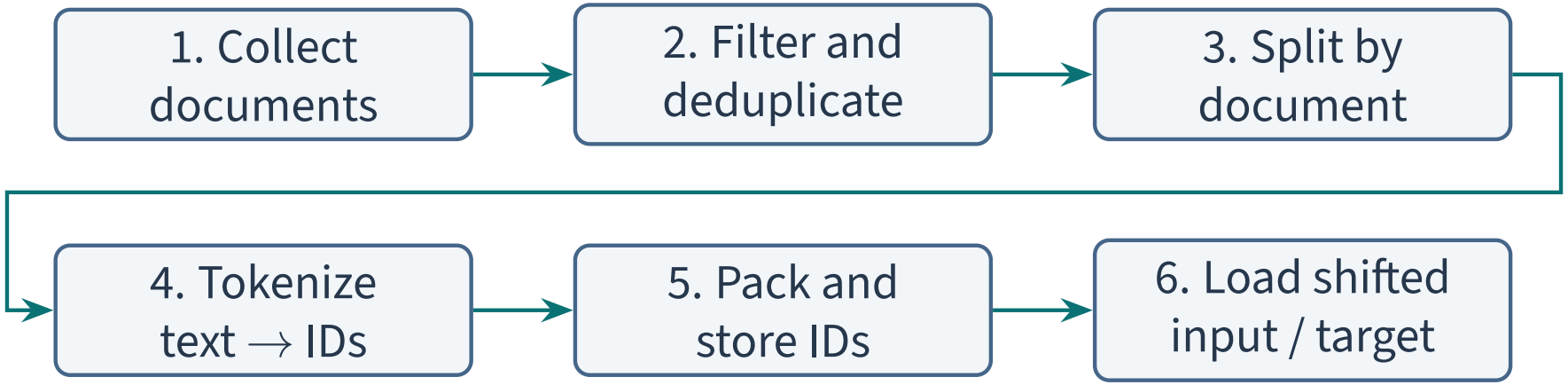
Guiding question

How do we turn raw documents into trustworthy next-token training batches?

What should be clear by the end?

1. Trace raw documents through filtering, tokenization, packing and batching.
2. Explain byte-level BPE and the vocabulary–sequence-length trade-off.
3. Pack documents while keeping boundaries explicit.
4. Verify binary storage, sampling, and the next-token shift.

The path from raw text to a training batch



Fit the tokenizer on the training split; process training and validation separately.

PART 1

From documents to tokens

1. From documents to tokens
2. Packing documents into examples
3. Storage and correctness

Start with documents, not model inputs

A document

A piece of text plus its source and identity: a book, article, web page, or other record.

Extract usable text

Keep content; remove or inspect navigation, markup, duplicated page furniture, and extraction errors.

Where might web text come from?

Web archives store captured pages and related records (WARC); extracted-text files (WET) provide a text-oriented starting point. Neither is automatically clean.

Filter, deduplicate, then split by document

1. **Filter:** inspect rules for empty, short, or repetitive text; record what they remove.
2. **Deduplicate:** group repeated documents before splitting to reduce train–validation leakage.
3. **Split:** assign documents to train or validation, then keep their streams separate.

Fit only on training data

Learn tokenizer merges on the training partition; apply the resulting tokenizer to both partitions.

Granularity moves cost between vocabulary and sequence

Unit	Vocabulary	Sequence	Main cost
bytes	256	very long	attention and context
words	millions	short	embeddings and unknowns
subwords	tunable	moderate	learned merge system

Why byte-level BPE?

Every UTF-8 string decomposes into 256 byte values, so the base system has no unknown text. Frequent byte sequences can then become single tokens.

System consequence

Tokenization changes sequence length, attention work, KV-cache size, multilingual efficiency, and the number of embedding parameters.

BPE learns a greedy compression hierarchy

1. Start from byte IDs and reserved special tokens.
2. Count adjacent token pairs in the training corpus.
3. Merge the most frequent pair into one new token.
4. Repeat until the vocabulary reaches its target size.

$$(p_0, p_1) \mapsto v_{\text{new}}, \qquad \text{repr}(v_{\text{new}}) = \text{repr}(p_0) \| \text{repr}(p_1)$$

Encoding new text

Apply learned merges by rank. Earlier learned pairs win when several merges are possible.

Source: Gage, Sennrich et al., GPT-2, and minBPE

A hand-worked merge exposes the algorithm

Corpus: `low low lower`

Round	frequent pair	stream fragment
0	–	`l o w | l o w | l o w e r`
1	`l o`	`lo w | lo w | lo w e r`
2	`lo w`	`low | low | low e r`

What the table hides

Real byte-level training also defines deterministic tie-breaking, special-token policy, and merge boundaries. These are part of the tokenizer artifact.

Pre-tokenization constrains where BPE can merge

Naive byte stream

Frequent punctuation and whitespace can merge across unrelated words or document structure.

Regex chunks

Letters, numbers, punctuation, contractions, and whitespace form separate regions. BPE merges stay inside each region.

`" the"` and `"the"` may become different tokens. GPT-2 uses pre-tokenization; the lab's simple BPE implementation does not.

Vocabulary size has two opposing effects

Larger vocabulary

fewer tokens per document
lower attention and decode length
better compression for frequent patterns

But also

larger embedding and output matrices
rarer tail tokens
more expensive vocabulary softmax

$$\text{embedding parameters} = Vd \qquad \text{compression} = \frac{\text{UTF-8 bytes}}{\text{tokens}}$$

Weight tying avoids paying Vd twice.

Bits per byte: compare on the same text

Two different questions

Tokenizer compression: how many tokens represent the text?

Model performance: how much probability does the model assign to it?

For the same held-out text, express negative log-likelihood in *bits per byte*:

$$\text{bits / byte} = \frac{\text{total negative log-likelihood (nats)}}{\ln 2 \times \text{UTF-8 bytes}} = \frac{\text{bits}}{\text{token}} \times \frac{\text{tokens}}{\text{byte}}.$$

Example

1,000 bytes, 400 tokens, summed negative log-likelihood 800 nats $\Rightarrow 1.15$ bits / byte.

Compare on identical text and boundary policy. Lower is better; fewer tokens alone do not guarantee a lower model loss.

Checkpoint: tokenizer choice

Think, then compare

A new tokenizer shortens a representative corpus by 20% but adds 300 million embedding parameters. Which training and serving measurements would decide whether this is a good trade?

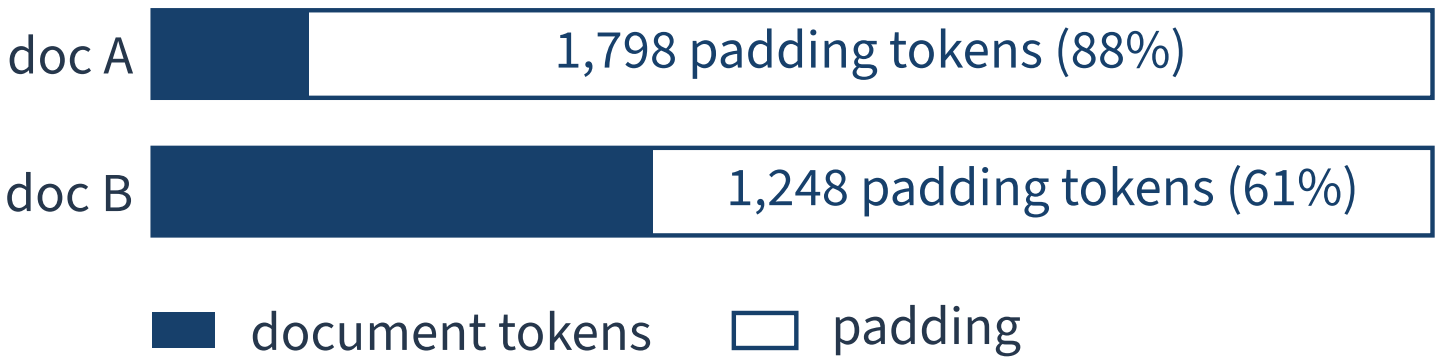
PART 2

Packing documents into examples

1. From documents to tokens
2. Packing documents into examples
3. Storage and correctness

Padding leaves unused positions

Suppose each example is padded to a 2,048-token window:



Packing

Concatenate documents with explicit boundaries and slice fixed-length windows. Padding overhead depends on the model and implementation.

Packing improves utilization but changes attention structure

```
[doc A][EOT][doc B][EOT][doc C]
```

- Every non-masked target can contribute to the loss.
- Windows can cross a document boundary.
- Standard causal attention lets doc B see doc A.

Two valid policies

Accept cross-document context and record it.

Or use block-diagonal causal masks with document offsets.

Invariant

Train and validation splits must happen at the document level before packing.

The end-of-text token has two roles

1. It preserves a recoverable boundary in a contiguous stream.
2. It teaches the model that one document ended and another may begin.

Special-token policy

Reserve its ID before training merges. Prevent ordinary text from being confused with the special marker. Test round-trip behavior explicitly.

PART 3

Storage and correctness

1. From documents to tokens
2. Packing documents into examples
3. Storage and correctness

Binary shards remove tokenization from the hot path

- Pre-tokenize once and store integer IDs contiguously.
- Use `uint16` only when every ID fits below $65{,}536$.
- Open shards with memory mapping and let the OS page cache serve workers.

Storage estimate

10^9 tokens at 2 bytes each occupy 2.0 GB before metadata.

Schema contract

Record dtype, tokenizer hash, token count, and document-boundary convention beside each shard.

One contiguous slice creates inputs and targets

Sample $T+1$ tokens beginning at offset i:

$$x = s[i : i+T] \qquad y = s[i+1 : i+T+1]$$

	0	1	2	$\cdots$	$T-1$
x	s_i	s_{i+1}	s_{i+2}	$\cdots$	s_{i+T-1}
y	s_{i+1}	s_{i+2}	s_{i+3}	$\cdots$	s_{i+T}

Fast correctness check

For every batch, $y[:, :-1]$ must equal $x[:, 1:]$ exactly.

A data pipeline needs end-to-end evidence

1. Encode and decode arbitrary Unicode text losslessly.
2. Verify deterministic token IDs and special-token handling.
3. Measure compression on representative languages and domains.
4. Audit document-level split membership before packing.
5. Measure loader throughput; check for GPU input stalls when a trace is available.

Do not optimize a corrupt stream

Decode sampled windows to inspect boilerplate and boundaries; compare document identities across splits to check leakage.

Checkpoint: pipeline audit

Think, then compare

A training run is fast, but validation loss is implausibly low. List three document- or token-level checks that distinguish a good model from split contamination.

Key takeaways

1. Byte-level BPE guarantees coverage, while merges trade vocabulary size for shorter sequences.
2. Pre-tokenization and special-token rules are part of the learned tokenizer.
3. Packing removes padding waste but requires an explicit document-boundary policy.
4. Binary shards are useful only when their schema, split, and target shift are verified.